অধ্যায় ২৮

Professional JavaScript Deployment, Docker, CI/CD, Cloud Infrastructure এবং Production Monitoring

শেখার লক্ষ্য

এই অধ্যায়ের শেষে development ও production environment-এর পার্থক্য, Twelve-Factor principles, reproducible build, container image, CI/CD, deployment strategies, health checks, observability, incident response এবং নিরাপদ rollback ব্যাখ্যা করতে পারবেন। **DeployReady Travel Operations API**-কে deployment-oriented structure-এ সাজাবেন।

২৮.১ Localhost থেকে Production: কী বদলায়?

Localhost-এ একটি process চালু হলেই কাজ হয়। Production-এ ব্যবহারকারী, network, DNS, TLS, secrets, persistent storage, uptime, monitoring, scaling এবং incident recovery বিবেচনা করতে হয়। Deployment শুধু code upload নয়; এটি একটি repeatable, reversible operational process।

Concern	Local development	Production expectation
Configuration	`.env` বা local variable	Managed configuration, secret store
Process	Terminal session	Supervised service/container
Data	Temporary/in-memory হতে পারে	Durable storage ও backup
Networking	localhost	TLS, ingress, firewall, DNS
Failures	Developer দেখে	Alerts, logs, incident process
Releases	Manual restart	CI, staged rollout, rollback

২৮.২ Twelve-Factor ধারণা ও Environment Configuration

Configuration code-এর বাইরে রাখতে হবে। `process.env.PORT` ব্যবহার করা যায়, কিন্তু value parse ও validate করতে হবে। Secret কখনো Git repository, frontend bundle বা Docker image-এ hardcode করা যাবে না। Development, staging ও production environment আলাদা হওয়া উচিত।

```javascript
const port = Number(process.env.PORT ?? 3000);
if (!Number.isInteger(port) || port < 0 || port > 65535) {
  throw new Error('Invalid PORT');
}
```

Important: `.env` ফাইল Git-এ commit না করলেও সেটি একা Secret Management নয়; production-এ access control, rotation ও audit দরকার।

২৮.৩ Build Artifact, Dependency Lock ও Reproducibility

Source code → automated checks → immutable build artifact → deployment। Dependency lockfile থাকলে `npm ci` deterministic dependency install করতে সাহায্য করে; তবে OS, runtime, native modules ও registry provenance-ও গুরুত্বপূর্ণ। এই demo-তে কোনো external runtime dependency নেই, তাই `npm install` প্রয়োজন হয় না। বাস্তব Express app-এ lockfile ও dependency scanning যুক্ত করতে হবে।

Release identity হিসেবে Git commit SHA এবং image digest সংরক্ষণ করা ভালো। `latest` tag একা rollback-এর জন্য যথেষ্ট নয়।

২৮.৪ Docker Fundamentals

Image হলো build করা filesystem ও metadata; container হলো image থেকে চালু হওয়া isolated process। Dockerfile-এর layer cache build দ্রুত করতে পারে। Non-root user, minimal base image, `.dockerignore`, health check ও explicit runtime command ব্যবহার করা ভালো।

```dockerfile
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY --chown=node:node package.json ./
COPY --chown=node:node src ./src
USER node
EXPOSE 3000
CMD ["node", "src/server.mjs"]
```

Security trade-off: Minimal image attack surface কমায়, কিন্তু image scanning, pinned digest, patched base image ও dependency review-এর বিকল্প নয়। Alpine-এ native dependency compatibility যাচাই করতে হয়।

২৮.৫ Docker Compose ও Local Production Simulation

`compose.yaml`-এ service, environment, ports, restart policy এবং shutdown grace period দেওয়া যায়। এই অধ্যায়ের demo-তে port শুধু `127.0.0.1`-এ bind করা হয়েছে, যাতে local machine-এর বাইরে অনিচ্ছাকৃত exposure কমে।

```yaml
services:
  api:
    build: .
    ports:
      - "127.0.0.1:3000:3000"
    environment:
      NODE_ENV: production
      PORT: 3000
    restart: unless-stopped
```

Docker Compose development ও small deployment-এ কার্যকর; multi-node orchestration, autoscaling, high availability-এর জন্য অতিরিক্ত infrastructure দরকার।

২৮.৬ CI: প্রতিটি Change যাচাই

Continuous Integration-এ code push বা pull request হলে syntax, unit/integration tests, type checks, lint, security checks এবং image build চালানো হয়। Fail করা test থাকলে release block করতে হবে।

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: npm test
      - run: docker build -t deployready:ci .
```

Security: Third-party actions production pipeline-এ full commit SHA দিয়ে pin করা, least-privilege permissions এবং secret exposure prevention বিবেচনা করুন। CI success মানে production readiness নয়।

২৮.৭ CD: Staging, Approval এবং Rollback

Continuous Delivery release-ready artifact তৈরি করে; Continuous Deployment সফল pipeline থেকে স্বয়ংক্রিয় release করে। একটি সাধারণ flow:

```
Pull Request → Tests → Build Image → Security Scan
     → Staging Deploy → Smoke Test → Approval
     → Production Rollout → Monitor → Rollback if needed
```

Blue/Green-এ দুটি environment থাকে; traffic switch করে দ্রুত rollback সম্ভব। Canary deployment-এ অল্প traffic নতুন version-এ পাঠানো হয়। Rolling deployment-এ instance ধাপে ধাপে বদলায়। **Database migration backward-compatible না হলে শুধু app rollback যথেষ্ট নাও হতে পারে।**

২৮.৮ Liveness, Readiness এবং Graceful Shutdown

`/health/live` process বেঁচে আছে কি না জানায়। `/health/ready` traffic গ্রহণের জন্য প্রস্তুত কি না জানায়। Readiness-এ dependency check থাকতে পারে, তবে costly query বা sensitive details প্রকাশ করা উচিত নয়। Shutdown-এর সময় প্রথমে readiness false, তারপর নতুন request বন্ধ ও চলমান কাজ শেষ করার চেষ্টা করা উচিত।

```javascript
process.on('SIGTERM', () => {
  app.drain();          // readiness becomes 503
  server.close(() => { process.exitCode = 0; });
});
```

এই snippet illustrative; complete timeout ও error handling `source-code/chapter-28/src/server.mjs`-এ রয়েছে। Container orchestration-এর termination grace period এবং application shutdown deadline সামঞ্জস্যপূর্ণ রাখতে হবে।

২৮.৯ Logs, Metrics এবং Traces

Observability-এর তিনটি গুরুত্বপূর্ণ signal: **Logs** (ঘটনার বিবরণ), **Metrics** (পরিমাপযোগ্য trend), **Traces** (request-এর end-to-end path)। Structured JSON log এবং request ID debugging সহজ করে। চারটি Golden Signals: latency, traffic, errors, saturation।

```javascript
console.log(JSON.stringify({ level: 'info', event: 'booking.created', requestId, bookingId }));
```

Logs-এ password, token, payment card বা personal data অপ্রয়োজনে রাখবেন না। Demo-এর `/metrics` JSON endpoint শিক্ষামূলক; এটি Prometheus exposition format নয় এবং production-এ publicly expose করা উচিত নয়।

২৮.১০ SLI, SLO এবং Alerting

SLI হলো observed indicator—যেমন successful requests ratio। SLO হলো target—যেমন rolling 30-day window-এ 99.9% successful requests। SLA হলো external contractual commitment। Alerts এমন হওয়া উচিত যাতে action নেওয়া যায়; প্রতিটি isolated error-এ paging করলে alert fatigue হয়।

উদাহরণ: ১০,০০০ request-এর মধ্যে ৯,৯৭০টি সফল হলে success rate = 99.7%। যদি SLO 99.9% হয়, target পূরণ হয়নি। Error budget ও traffic window ছাড়া শুধু একটি percentage বিভ্রান্তিকর হতে পারে।

২৮.১১ Secrets, Supply Chain ও Infrastructure Security

Production deployment-এ TLS termination, network segmentation, restricted database access, secret manager, container scanning, SBOM, dependency review এবং least-privilege CI credentials দরকার। Deployment key বা cloud credential source code-এ রাখা যাবে না। Rotatable short-lived credentials ও

workload identity উত্তম।

২৮.১২ Backups, Disaster Recovery ও Incident Response

RPO (Recovery Point Objective) হলো কতটা data loss সহ্য করা যাবে; RTO (Recovery Time Objective) হলো কত সময়ের মধ্যে service ফিরিয়ে আনতে হবে। Backup আছে মানেই restore কাজ করে—এমন নয়; restore drill করতে হবে। Incident runbook-এ detection, escalation, mitigation, communication, rollback এবং postmortem থাকা উচিত।

২৮.১৩ Practical Project — DeployReady Travel Operations API

এই Project-এ Node.js built-in HTTP server দিয়ে একটি ছোট Travel Operations Jobs API বানানো হয়েছে। Production concepts শেখার জন্য intentionally dependency-free design। **এটি বাস্তব travel booking/payment backend নয়।**

Project Structure

```
source-code/chapter-28/
├── package.json
├── Dockerfile
├── .dockerignore
├── compose.yaml
├── .github/workflows/ci.yml
├── src/app.mjs
├── src/server.mjs
├── test/app.test.mjs
└── README.md
```

API Contract

Method	Route	Purpose
GET	`/health/live`	Process liveness
GET	`/health/ready`	Traffic readiness
GET	`/metrics`	Educational counters
GET	`/api/jobs`	In-memory job list
POST	`/api/jobs`	Validated job creation

Run and Verify

```
TERMINAL
cd source-code/chapter-28
npm test
npm start
# separate terminal
curl http://localhost:3000/health/ready
curl -X POST http://localhost:3000/api/jobs \
  -H 'content-type: application/json' \
  -d '{"title":"Verify hotel booking"}'
```

Docker installed থাকলে `docker compose up --build` দিয়ে চালাতে পারবেন। CI workflow GitHub repository-তে push করলে চলবে; এই environment-এ GitHub Actions run করা হয়নি।

Design Decisions

- `app.mjs`-এ HTTP handler ও state; `server.mjs`-এ process lifecycle। এতে test-এ random port ব্যবহার করা যায়।
- `x-request-id` response header request correlation-এর পরিচিতি দেয়।
- Payload size limit ও title validation আছে; কিন্তু authentication, durable database এবং rate limiting নেই।
- `/health/ready` shutdown-এ 503 দেয়; `/health/live` process active থাকা পর্যন্ত 200 দেয়।
- In-memory data restart-এ হারিয়ে যাবে। Production deployment-এর আগে persistent database, access control, secret management ও observability দরকার।

২৮.১৪ Common Deployment Mistakes

1. `.env` বা private key Git-এ commit করা।
2. Image tag `latest` ব্যবহার করে কোন version deploy হয়েছে তা না জানা।
3. Database migration rollback compatibility পরীক্ষা না করা।
4. Readiness ও liveness একইভাবে implement করা।
5. No timeout, no graceful shutdown।
6. শুধু HTTP 200 দেখে success ধরে নেওয়া; latency ও error rate না দেখা।
7. In-memory session ব্যবহার করে multiple replicas চালানো।
8. Test ছাড়াই production release করা।
9. Alerting তৈরি করে runbook না রাখা।
10. Backup তৈরি করেও restore test না করা।

২৮.১৫ Exercises

১. Development ও production-এর পাঁচটি পার্থক্য লিখুন।
২. `PORT=abc` হলে কেন fail-fast করা উচিত ব্যাখ্যা করুন।
৩. Docker image ও container-এর পার্থক্য লিখুন।

৪. `.dockerignore`-এ `node_modules` রাখার কারণ কী?

৫. Liveness ও readiness-এর আলাদা উদ্দেশ্য উদাহরণসহ লিখুন।

৬. `POST /api/jobs`-এ empty title পাঠিয়ে status code যাচাই করুন।

৭. Request ID কীভাবে incident debugging-এ সাহায্য করে?

৮. Blue/Green ও Canary deployment তুলনা করুন।

৯. ৪,০০০ request-এর মধ্যে ৩,৯৮০ সফল হলে success rate গণনা করুন।

১০. DeployReady-তে persistent database যোগ করার migration plan লিখুন।

Sample Solutions

১. Config, storage, network, process supervision ও monitoring—প্রতিটি production-এ অধিক নিয়ন্ত্রিত।

২. Invalid port দিয়ে অনির্দেশ্য behaviour হওয়ার চেয়ে startup-এ clear error নিরাপদ।

৩. Image immutable template; container তার running instance।

৪. Host-এর dependency, unnecessary size এবং platform mismatch এড়াতে।

৫. Alive কিন্তু DB unavailable হলে live=200, ready=503 হতে পারে।

৬. `curl -i -X POST http://localhost:3000/api/jobs -d '{"title":""}'` → 422; content-type সহ JSON পাঠানো উত্তম।

৭. একই ID দিয়ে reverse proxy, application ও tracing events correlate করা যায়।

৮. Blue/Green দুই environment switch করে; Canary ছোট traffic slice দিয়ে শুরু করে।

৯. `3980 / 4000 * 100 = 99.5%`।

১০. Repository abstraction, schema migration, connection secret, readiness dependency, integration tests, backup ও rollback plan যুক্ত করুন।

২৮.১৬ MCQ — Final Check

১. Readiness failure-এ সাধারণত কী হয়? A) traffic বন্ধ B) CPU double C) image delete D) DNS reset

২. `SIGTERM` সাধারণত কী signal? A) shutdown request B) SQL error C) test pass D) port scan

৩. Container image কী? A) build artifact B) live database C) monitoring alert D) browser tab

৪. CI-এর উদ্দেশ্য কী? A) automatic checks B) শুধু CSS C) manual backup D) invoice

৫. Secret কোথায় রাখা উত্তম? A) managed secret store B) public Git C) frontend JS D) image label

৬. SLO কী? A) target objective B) request body C) CSS property D) Docker network

৭. Canary rollout-এ কী করা হয়? A) limited traffic B) delete tests C) disable logs D) expose secrets

৮. RPO কী বোঝায়? A) tolerable data loss B) uptime port C) memory speed D) request ID

৯. Request ID-এর সুবিধা? A) correlation B) encryption C) authorization D) compression

১০. Production-এ in-memory data-এর ঝুঁকি? A) restart data loss B) faster TLS C) immutable DB D) automatic backup

Answers ও ব্যাখ্যা: ১-A (unready instance-এ নতুন traffic না দেওয়া), ২-A (graceful termination trigger), ৩-A (container চালানোর template), ৪-A (repeatable validation), ৫-A (controlled access/rotation), ৬-A (service-level target), ৭-A (small cohort-এ risk সীমিত), ৮-A (maximum acceptable data loss), ৯-A (distributed debugging), ১০-A (durability নেই)।

২৮.১৭ Chapter Recap ও Next Step

Production readiness মানে শুধু server চালু থাকা নয়; reproducible builds, validated releases, security, health signals, observability, durable data এবং incident recovery একসঙ্গে কাজ করা। পরবর্তী অধ্যায়ে microservices, distributed transactions ও system design-এর trade-offs আলোচনা করব।

Further reading: Node.js HTTP/Process documentation, Docker Dockerfile best practices, GitHub Actions security hardening, OpenTelemetry concepts, Google SRE workbook। Version-specific details অফিসিয়াল documentation দিয়ে যাচাই করতে হবে।